

Python Review 1 — Lists, Arrays, Functions & Basic Plotting (10 s)

> Overview

↳ 2 cells hidden

Task 1:

1. In the **Markdown** cell below this cell, enter your name on the line with level 3 heading (###)
2. In the **code** cell below the cell with your name, code and execute a line in python that prints `"Hello World"`



Jonah Ebent

```
print('Hello world!')
```

```
Hello world!
```

Task 2: Lists vs NumPy Arrays

Goal: build the same `x` values as a **Python list** and as a **NumPy array**.

1. Run the example below.
2. Notice the output of `len(...)` vs `.shape` for each object.
3. Notice indexing looks the same (`x[0]`), but behavior differs for math.

```
import math
import numpy as np
import matplotlib.pyplot as plt

# Example range of x values (in radians)
x_list = [0.0, 0.5, 1.0, 1.5, 2.0]
x_arr = np.array(x_list)

print("x_list:", x_list)
print("len(x_list):", len(x_list))
print("x_arr:", x_arr)
print("x_arr.shape:", x_arr.shape)

print("First element list:", x_list[0])
print("First element array:", x_arr[0])
```

```
x_list: [0.0, 0.5, 1.0, 1.5, 2.0]
len(x_list): 5
x_arr: [0.  0.5 1.  1.5 2. ]
x_arr.shape: (5,)
First element list: 0.0
First element array: 0.0
```

Q1 - What is the difference between a Python list and a NumPy array?

Insert a Markdown cell below this cell and answer the question.



Python lists are strictly one-dimensional. They are also native to Python (i.e., they don't need a separate module).

NumPy arrays can be n dimensions. They are also not native to Python but require importing the `numpy` module. NumPy arrays are good for doing matrix math and manipulations.

Task 3 — Evaluate and plot variations of the Sine Function

Evaluate and plot $y=\sin(x)$, $y=\sin(2x)$, and $y=\sin(x + \pi/2)$ using **Python lists and NumPy arrays**.

Read the code carefully to review:

- Where loops are needed and where they are not
- How the same function can be implemented in different ways

```
# Evaluating a function using a NumPy array and vectorized operations
```

```
import numpy as np
import math
```

```
# Create x values as a NumPy array over one period of the wave
x_array = np.linspace(0, 2 * math.pi, 50)
```

```
# Evaluate sin(x) using NumPy's vectorized function
y_sin_array = np.sin(x_array)
```

```
# Print a few values to verify the computation
print("First five x values:", x_array[:5])
print("First five sin(x) values:", y_sin_array[:5])
```

```
First five x values: [0.          0.12822827 0.25645654 0.38468481 0.51291309]
First five sin(x) values: [0.          0.12787716 0.25365458 0.375267    0.49071755]
```

```
# Evaluating a function using a Python list and a for loop
```

```
# Define a function that computes sin(2x) for a single value
def sin_2x(x):
    return math.sin(2 * x)
```

```
# Create x values as a Python list
x_list = list(np.linspace(0, 2 * math.pi, 50))
```

```
# Use a loop to evaluate the function for each x
y_sin_2x_list = []
```

```
for x in x_list:
    y_sin_2x_list.append(sin_2x(x))
```

```
# Print a few values to verify the computation
print("First five sin(2x) values:", y_sin_2x_list[:5])
```

```
First five sin(2x) values: [0.0, 0.25365458390950735, 0.49071755200393785, 0.6956825506034864, 0.8551427630053461]
```

```
# Define a function that works on NumPy arrays
def sin_shifted(x):
    return np.sin(x + math.pi / 2)
```

```
# Reuse the NumPy x array
y_sin_shifted_array = sin_shifted(x_array)
```

```
# Print a few values to verify the computation
print("First five sin(x + pi/2) values:", y_sin_shifted_array[:5])
```

```
First five sin(x + pi/2) values: [1.          0.99179001 0.96729486 0.92691676 0.8713187 ]
```

Create simple plots to visualize your 3 functions

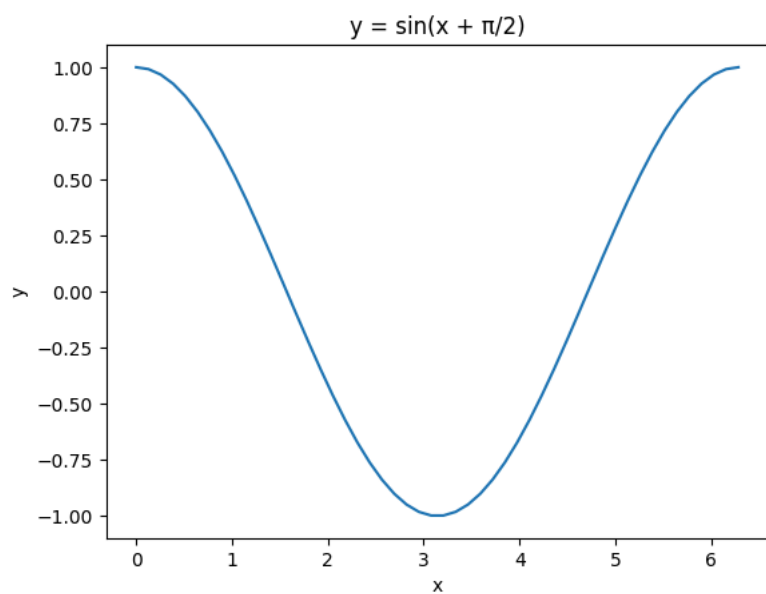
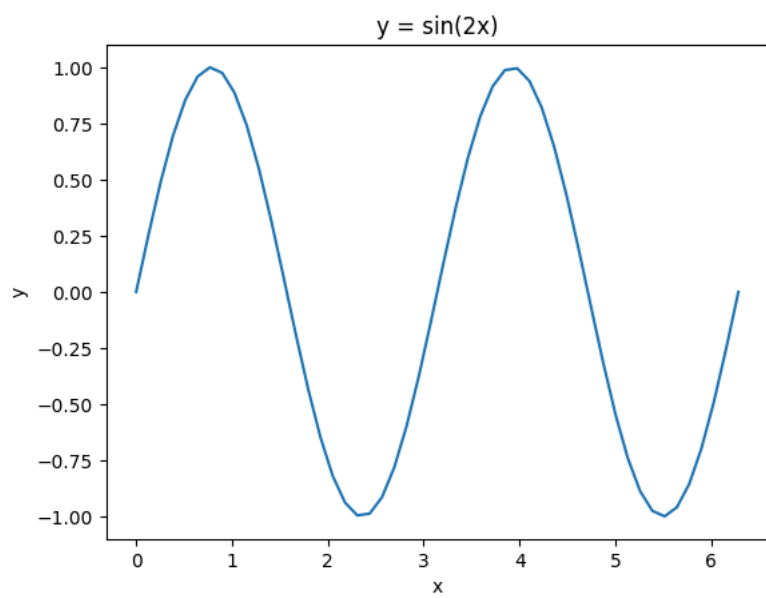
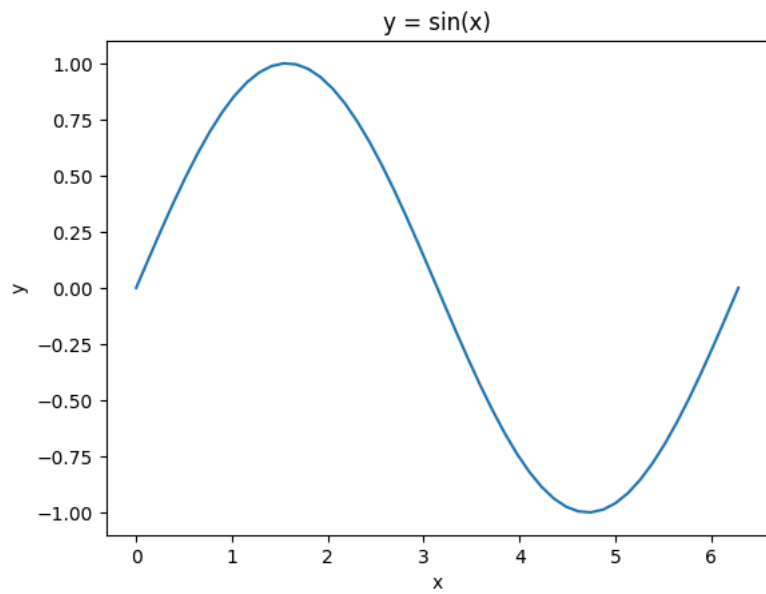
```
import matplotlib.pyplot as plt
```

```
# Plot sin(x)
plt.figure()
plt.plot(x_array, y_sin_array)
plt.title("y = sin(x)")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

```
# Plot sin(2x)
```

```
plt.figure()
plt.plot(x_list, y_sin_2x_list)
plt.title("y = sin(2x)")
plt.xlabel("x")
plt.ylabel("y")
plt.show()

# Plot sin(x + pi/2)
plt.figure()
plt.plot(x_array, y_sin_shifted_array)
plt.title("y = sin(x +  $\pi/2$ )")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



- ✓ This is how you make a fancier plot with all your functions on it

```

plt.figure(figsize=(9, 5))

plt.plot(x_array, y_sin_array,
         label="sin(x)",
         linewidth=2)

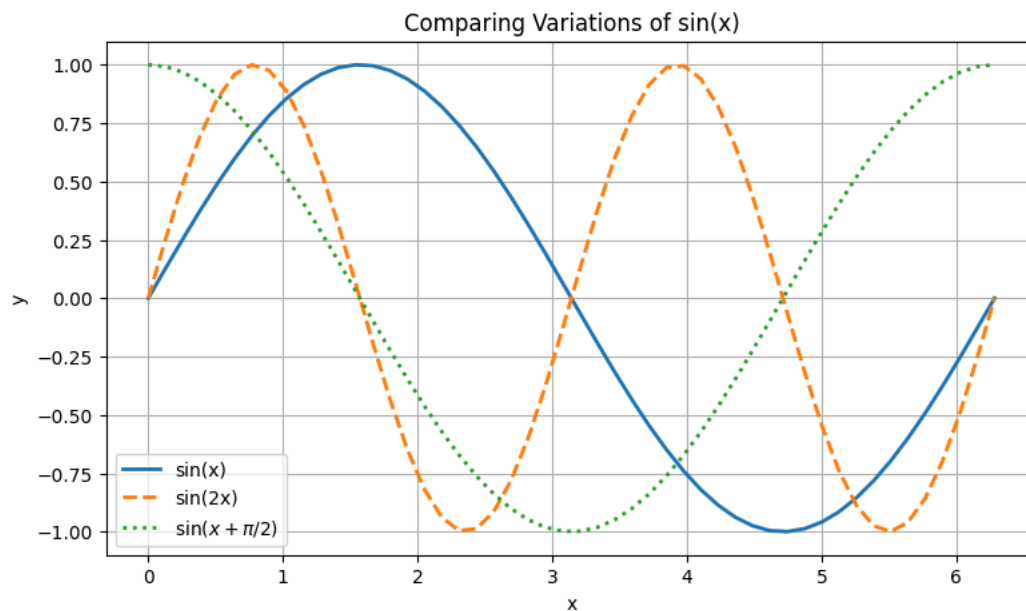
plt.plot(x_list, y_sin_2x_list,
         linestyle="--",
         label="sin(2x)",
         linewidth=2)

plt.plot(x_array, y_sin_shifted_array,
         linestyle=":",
         label=r"y=sin(2x)",
         linewidth=2)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Comparing Variations of sin(x)")
plt.legend()
plt.grid(True)

plt.savefig("sin_variations.png", dpi=150, bbox_inches="tight")
plt.show()

```



✓ Task 4: Define and evaluate $g(x) = \cos(x)$

1. Write a function $g(x)$ that returns $\cos(x)$ using the `math` library.
2. Evaluate $g(x)$ for every value in `x_list` using a loop.
3. Store results in `y_g_list`.



```

def g(x: float) -> float:
    """Returns cos(x) for a single number x (radians)."""
    return math.cos(x)

y_g_list = []

for x in x_list:
    y_g_list.append(g(x))

print("y_g_list:", y_g_list)

```

```
y_g_list: [1.0, 0.9917900138232462, 0.9672948630390295, 0.9269167573460217, 0.8713187041233894, 0.8014136218679567, 0.7183493500]
```

Task 5: Plot $\sin(x)$ and $\cos(x)$



1. Make **one plot** with *both* $\sin(x)$ and $\cos(x)$ on the same axes (with legend, title, labels).
2. Change the color and line type of each plot to something different from the defaults.

Consult the matplotlib documentation if needed:

<https://matplotlib.org/stable/contents.html>

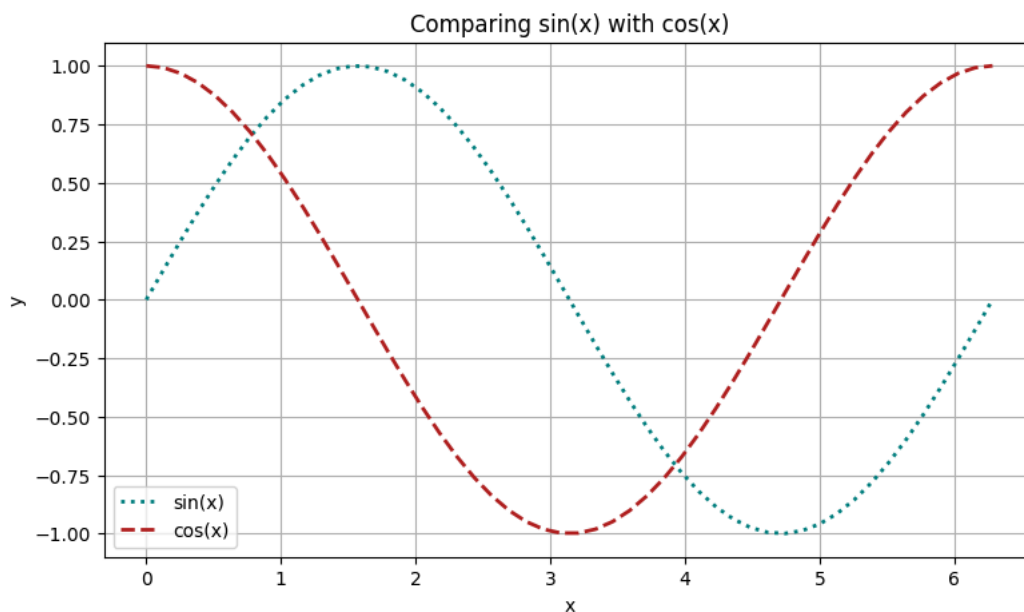
```
# TODO: Combined plot
plt.figure(figsize=(9, 5))

plt.plot(x_array, y_sin_array,
         label="sin(x)",
         linestyle=':',
         color='teal',
         linewidth=2)

plt.plot(x_list, y_g_list,
         label="cos(x)",
         linestyle="--",
         color='firebrick',
         linewidth=2)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Comparing sin(x) with cos(x)")
plt.legend()
plt.grid(True)

plt.savefig("sin_cos_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```



Task 6: Linear vs Quadratic

Choose an x range and compute:

- A linear function (example form): $y=\sin(2x)$
- A quadratic function (example form): $y=\sin(x + \pi/2)$

Requirement

- Use **lists + loops** for **one** of these.
- Use **NumPy arrays** for the **other**.



Then **plot both** functions (your choice: same axes or separate).



```
# Choose your x-range
x_list2 = np.linspace(0, 100)

x_arr2 = np.array(x_list2)

# Pick coefficients (choose simple numbers)
a = 1.0
b = 2.0
c = -5000.0

def linear(x):
    return a * x + b

def quadratic(x):
    return a * x ** 2 + b * x + c

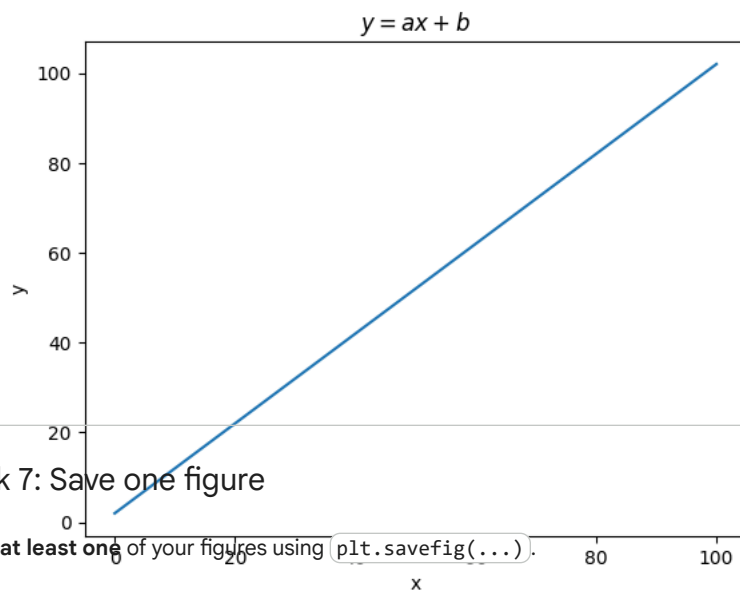
# Just so you know, this works too:
# y_linear_list = list(map(linear, x_list2))

y_linear_list = []
for x in x_list2:
    y_linear_list.append(linear(x))

y_quad_arr = quadratic(x_arr2)

# Plot linear function
plt.figure()
plt.plot(x_list2, y_linear_list)
plt.title("y=sin(2x)")
plt.xlabel("x")
plt.ylabel("y")
plt.show()

# Plot quadratic function
plt.figure()
plt.plot(x_list2, y_quad_arr)
plt.title("y=sin(x + \pi/2)")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



Task 7: Save one figure

Save at least one of your figures using `plt.savefig(...)`.



Filename convention: `lastname_review1.png` (replace `lastname` with your last name)

```
plt.figure(figsize=(9, 5))

plt.plot(x_list2, y_linear_list,
         label=f"y=sin(2x)",
         linestyle=':',
         color='teal',
         linewidth=2)

plt.plot(x_list2, y_quad_arr,
         label=f"y=sin(x + \pi/2)",
         linestyle="--",
         color='firebrick',
         linewidth=2)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Comparing linear with quadratic functions")
```